

Mentorship Management System

A MERN Stack Web Application for Student and Faculty
Mentoring

PROJECT REPORT

Submitted to: Jeelani Asif

This document describes the problem, design, architecture, and complete implementation of the Mentorship Management System.

Contents

1	Introduction	3
1.1	Overview	3
1.2	Motivation	3
1.3	Problem Statement	4
1.4	Objectives	4
1.5	Scope of the Project	4
1.6	Report Organization	4
2	Technology Stack	5
2.1	Backend	5
2.2	Frontend	5
2.3	Why This Stack	6
3	System Architecture	6
3.1	High-Level Architecture	6
3.2	Request Flow	6
3.3	Folder Structure	7
4	Use Case Analysis	8
4.1	Actors	8
4.2	Use Case Diagram	9
4.3	Use Case Descriptions	9
5	Database Design	10
5.1	Entity Overview	10
5.2	Entity Relationship Diagram	11
5.3	Relationship Notes	12
6	Modules and Features	12
6.1	Authentication and User Management	12
6.2	Student Module	12
6.3	Faculty Module	13
6.4	Admin Module	13
6.5	Meetings, Notes, and Feedback	13
7	Sequence Diagrams	14
7.1	User Login	14
7.2	Mentor Request and Approval	14

7.3	Meeting and Feedback	14
8	API Reference	14
8.1	Conventions	15
8.2	Endpoint Summary	15
8.3	Sample Request and Response	16
9	Security Considerations	17
9.1	Implemented	18
9.2	Recommended Hardening	18
10	Testing	18
10.1	Approach	19
10.2	What Was Tested	19
10.3	Manual Frontend Verification	19
11	Conclusion and Future Scope	19
11.1	Current Status	20
11.2	Key Outcomes	20
11.3	Future Scope	20

Chapter 1

Introduction

1.1 Overview

The Mentorship Management System is a full-stack web application built to digitize and streamline the student and faculty mentorship process within an academic institution. Traditionally, mentor allocation, meeting scheduling, and feedback collection in most institutes are handled manually through registers, spreadsheets, or informal communication. This creates gaps in accountability, makes it difficult for administrators to track coverage across departments, and leaves no structured history of mentoring interactions.

This system addresses that gap by providing a single, structured platform that supports three distinct user roles: Student, Faculty (acting as a Mentor), and Admin. Each role is given a dedicated dashboard and a well-defined set of permissions, enforced through authenticated API access.

The application is built using the MERN stack, an acronym for MongoDB, Express.js, React, and Node.js. The backend exposes a RESTful JSON API, and the frontend is a single-page application built with React and Vite that consumes this API over HTTPS.

1.2 Motivation

Mentorship is a critical support structure in engineering education. Students benefit from regular, recorded interaction with a faculty mentor who can guide them on academics, projects, and general well-being. However, without a system of record, several practical problems emerge:

- Mentor allocation is done manually, department by department, and is slow to update when a mentor is overloaded or unavailable.
- There is no single place to see which students have not yet been assigned a mentor.
- Meetings are agreed upon informally and are easy to forget, miss, or leave unrecorded.
- Session outcomes and notes are not retained anywhere searchable.
- There is no closed feedback loop that lets an administrator judge whether mentor-

ship is actually working.

The project was undertaken to solve these problems with a concrete, working prototype rather than a theoretical design, and the resulting system is functional end to end across all three roles.

1.3 Problem Statement

Design and implement a role-based web application that allows students to request and be assigned a faculty mentor, allows faculty to approve requests and manage their mentees, allows both parties to schedule and record mentorship meetings with notes and feedback, and allows an administrator to oversee and reassign mentorship allocations across departments, all while enforcing secure, role-appropriate access to every operation.

1.4 Objectives

- Provide role-based access and dashboards for Students, Faculty, and Admins.
- Allow students to request mentor allocation and allow faculty to review and approve those requests.
- Enable scheduling, tracking, updating, and cancellation of mentorship meetings.
- Allow faculty to record structured session notes against a specific meeting.
- Collect structured feedback, a numeric rating together with an optional comment, from students after a meeting.
- Give administrators the ability to reassign a student's mentor and view a department-wise summary of mentorship activity.
- Secure every state-changing operation using JWT-based authentication and server-side role authorization, rather than relying on the frontend to hide options.

1.5 Scope of the Project

This report documents the current, working implementation of the system: a REST API covering seven resource domains (users, mentors, students, assignments, meetings, notes, and feedback), a React single-page application with dedicated dashboards for all three roles, JWT-based authentication with role authorization middleware, and MongoDB persistence through Mongoose schemas. It is a complete functional prototype, suitable for demonstration, evaluation, and further iterative hardening toward production use, and the report is written to reflect exactly what is implemented in the codebase rather than an idealized design.

1.6 Report Organization

Chapter 2 describes the technology stack and the reasoning behind it. Chapter 3 covers system architecture and request flow. Chapter 4 covers the actors and use cases of the system. Chapter 5 documents the database design and entity relationships. Chapter 6 walks through each functional module. Chapter 7 presents sequence diagrams for the three principal workflows. Chapter 8 is the complete API reference. Chapter 9 discusses security measures and hardening recommendations. Chapter 10 covers the testing approach used during development. Chapter 11 concludes the report and outlines future scope.

Chapter 2

Technology Stack

2.1 Backend

Component	Technology
Runtime	Node.js
Web Framework	Express.js (v5)
Database	MongoDB (Atlas), accessed via the Mongoose ODM
Authentication	JSON Web Tokens, via the <code>jsonwebtoken</code> package
Password Hashing	<code>bcryptjs</code>
Cross-Origin Support	<code>cors</code>
Environment Configuration	<code>dotenv</code>
Development Tooling	<code>nodemon</code> for hot-reloading the API server

2.2 Frontend

Component	Technology
Library	React 19
Build Tool	Vite
Routing	React Router DOM v7
HTTP Client	Axios
Linting	Oxlint

Component	Technology
State Management	React Context API, through a custom <code>AuthContext</code>

2.3 Why This Stack

The MERN stack was selected for several concrete reasons relevant to this project:

- **Single language across the stack.** Both the client and server are written in JavaScript, which reduced context-switching during development and allowed data structures such as the assignment and meeting objects to be reasoned about consistently on both sides.
- **Fast iteration.** Vite's hot module reload and `nodemon` on the backend made the feedback loop for UI and API changes very short during development.
- **Schema flexibility.** MongoDB's document model, combined with Mongoose schemas for validation, suited mentorship data that evolves over time, such as adding new assignment statuses or optional fields, without heavy migration overhead.
- **Stateless authentication.** JWTs allow the API to authenticate every request independently, without server-side session storage, which keeps the backend simple to reason about and easy to scale horizontally.

Chapter 3

System Architecture

3.1 High-Level Architecture

The system follows a three-tier architecture: a React single-page application on the client, a stateless Express REST API in the middle tier, and MongoDB as the persistence layer. Figure 3.1 shows the overall shape of the system, including where authentication is enforced.

3.2 Request Flow

Every protected request travels through the same pipeline before reaching business logic:

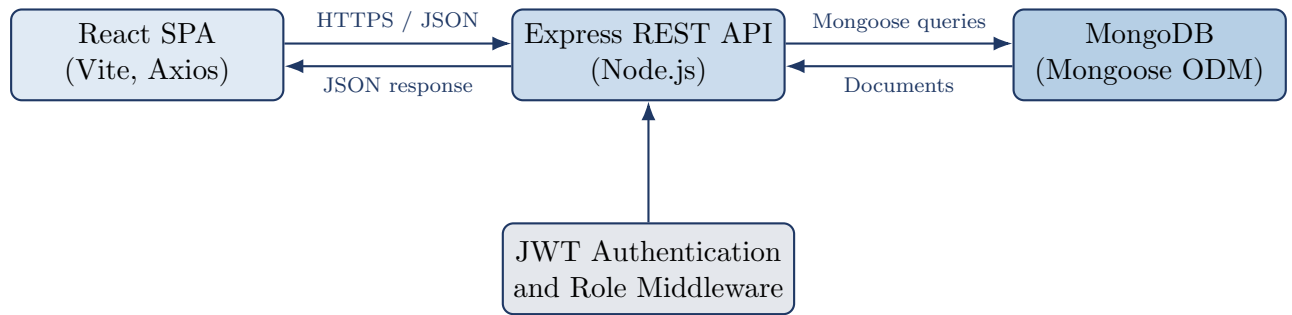


Figure 3.1: Three-tier architecture of the Mentorship Management System

1. The React client sends an HTTP request through Axios to an Express route. For protected endpoints, it attaches a JWT in the **Authorization: Bearer <token>** header. The token was issued at login and is stored on the client after authentication.
2. The **protect** middleware reads the header, verifies the token's signature using the server's JWT secret, and, if valid, decodes the user's identity and role onto **req.user**. If the token is missing or invalid, the middleware short-circuits the request with a 401 response.
3. The **authorizeRoles(...)** middleware checks that **req.user.role** is in the list of roles permitted for that route. For example, only a user with the **faculty** role may call the mentor-approval endpoint, and only **admin** may call the reassignment endpoint. A user with the wrong role receives a 403 response.
4. The matching controller function executes the business logic for that operation, reading and writing data through the relevant Mongoose model or models.
5. A JSON response with a **success** flag, a human-readable **message**, and, where applicable, a **data** payload is returned to the client.

3.3 Folder Structure

The backend and frontend are organized as two separate projects, connected only through the REST API contract. This separation allows the two halves to be developed, tested, and deployed independently.

Listing 3.1: Backend folder structure

```

1 mentorship-backend/
2   src/
3     config/db.js           # MongoDB connection setup
4     controllers/           # Business logic per resource
5     middleware/             # Auth and error handling
6     models/                 # Mongoose schemas
7     routes/                 # Express routers per resource
8     utils/asyncHandler.js
9     server.js               # Application entry point
  
```

Listing 3.2: Frontend folder structure

```

1 mentorship-frontend/
2   src/
3     components/           # Navbar, Sidebar, Modal, StatCard
4       , and others
5     context/AuthContext.jsx # Authentication state provider
6     pages/
7       admin/              # AdminDashboard,
8         DepartmentSummary, ReassignMentee
9       faculty/            # FacultyDashboard, Mentees,
10        Availability, AddNote
11      student/            # StudentDashboard, RequestMentor
12        , BookMeeting
13      Login.jsx, Register.jsx
14      services/api.js      # Configured Axios instance and
15        API calls

```

Chapter 4

Use Case Analysis

4.1 Actors

The system has three primary actors, each mapped directly to the `role` field on the `User` model.

Actor	Description
Student	A registered student who can request a mentor, book meetings with their assigned mentor, view their meeting history, and submit feedback.
Faculty	A registered faculty member acting as a mentor. Can review and approve incoming mentor requests, manage availability, view assigned mentees, and add notes to meetings.
Admin	An administrative user with system-wide visibility. Can view department-level summaries and reassign a student from one mentor to another.

4.2 Use Case Diagram

Figure 4.1 groups each actor with the use cases specific to that role. All three roles authenticate through the same registration and login screen; this shared entry point is shown once, under the Student actor, and applies equally to Faculty and Admin.

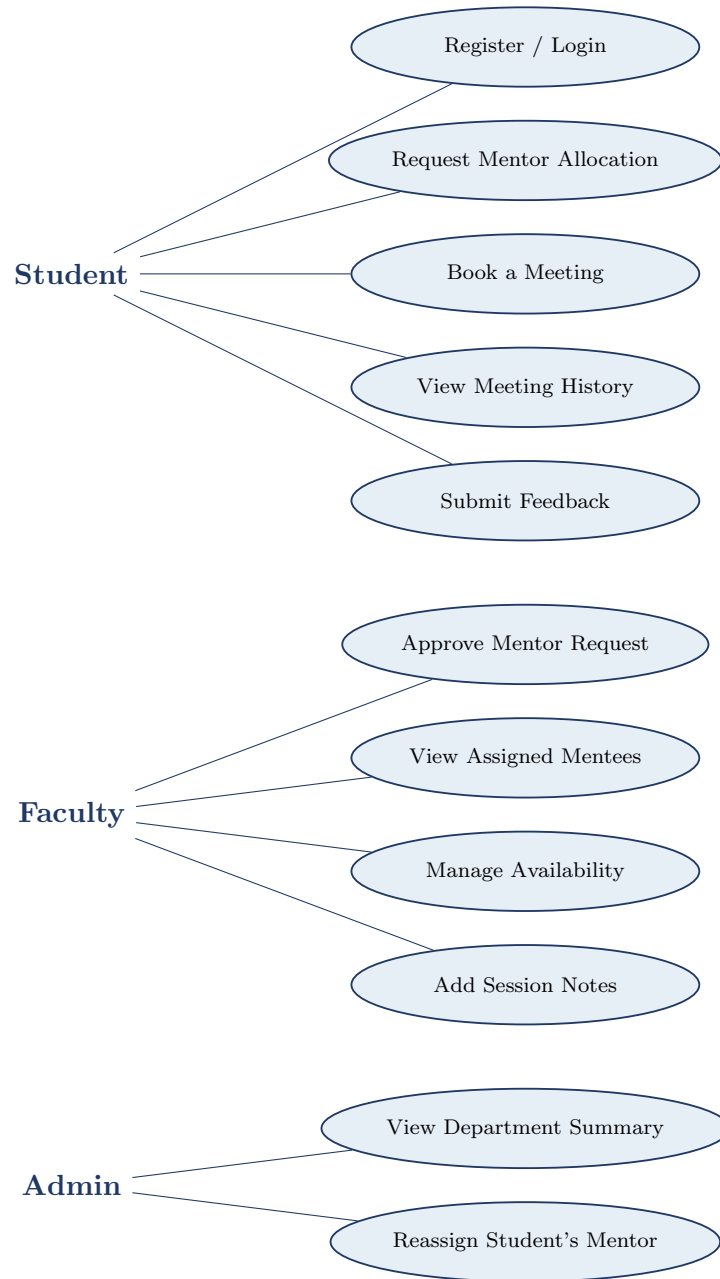


Figure 4.1: Use case diagram showing actor to use case relationships

4.3 Use Case Descriptions

UC: Request Mentor Allocation. A student with no active mentor submits a request, optionally naming a preferred faculty member. The system rejects the request if the

student already has a pending request. The request is stored as an **Assignment** document with status **pending**.

UC: Approve Mentor Request. A faculty member views all pending requests, then approves one. On approval, the assignment status changes to **active**, the faculty's identity is attached to the assignment, and the student's own **mentor** reference is updated to point at that faculty member's mentor profile.

UC: Reassign Mentor. An admin selects an existing assignment and specifies a new faculty member. The system verifies the target user exists and holds the **faculty** role, updates the assignment, sets its **reassignedFlag** to true, and updates the student's mentor reference to match.

UC: Book Meeting. A student with an active mentor schedules a meeting for a chosen date, with an optional agenda. The meeting starts in **scheduled** status and can later be marked **completed** or **cancelled**.

UC: Submit Feedback. After a meeting, a student submits a rating from 1 to 5 together with an optional free-text comment, tied to that specific meeting.

Chapter 5

Database Design

5.1 Entity Overview

The system uses seven core Mongoose collections. Relationships between them are expressed through **ObjectId** references rather than embedded documents, which keeps each collection independently queryable and avoids duplicating data such as user profiles across records.

Model	Purpose and Key Fields
User	The base account for every person in the system: name, email (unique), a bcrypt-hashed password, a role of student , faculty , or admin , and a department. All authentication is performed against this collection.
Student	Extends a User with a unique roll number, department, semester (1 to 12), and a reference to the currently assigned Mentor .

Model	Purpose and Key Fields
Mentor	Extends a User with department, a list of expertise tags, and a free-text availability field.
Assignment	The link between a student and a faculty mentor. Tracks a status of <code>pending</code> , <code>active</code> , <code>reassigned</code> , or <code>completed</code> , the date assigned, and a boolean <code>reassignedFlag</code> that records whether the student has been moved from their original mentor.
Meeting	A scheduled mentor and student meeting: date, an optional agenda, a status of <code>scheduled</code> , <code>completed</code> , or <code>cancelled</code> , and a notes field.
Note	A standalone note tied to a specific meeting through <code>meetingId</code> , used for more detailed session records than the meeting's own notes field.
Feedback	Student feedback tied to a specific meeting: a required rating from 1 to 5 and an optional comment.

All seven schemas use Mongoose's `timestamps` option, so every document automatically records `createdAt` and `updatedAt`, which supports both the department summary statistics and general auditability.

5.2 Entity Relationship Diagram

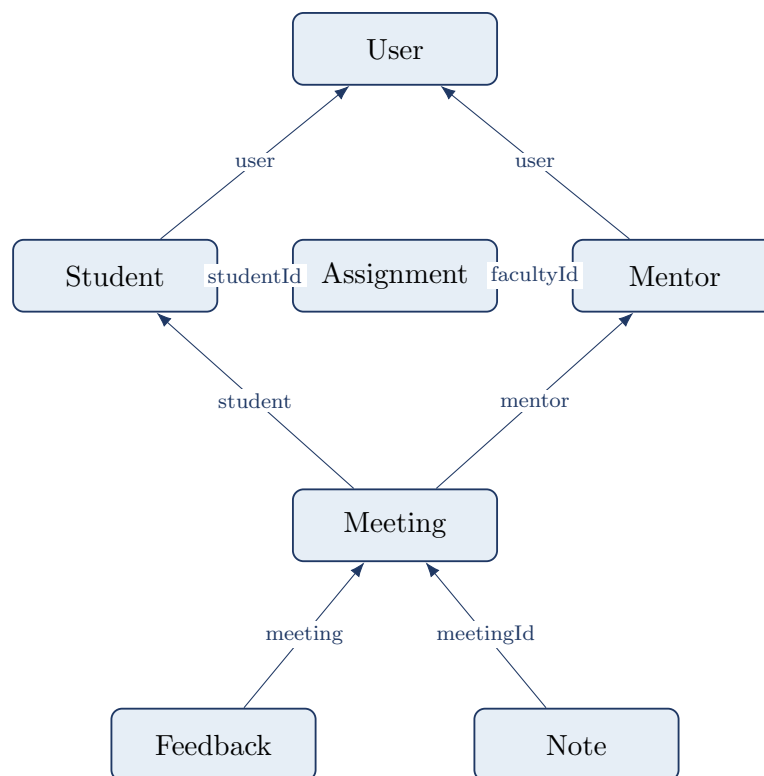


Figure 5.1: Entity relationship diagram of the seven core collections

5.3 Relationship Notes

- A `Student` and a `Mentor` document each reference exactly one underlying `User`, which keeps authentication data separate from role-specific profile data.
- An `Assignment` is the record of truth for who is currently mentoring whom, while the `Student.mentor` field is a denormalized pointer kept in sync whenever an assignment is approved or reassigned, so that mentor lookups for a student do not require querying the assignment collection first.
- A `Meeting` references both a `Mentor` and a `Student` directly, so meeting history can be queried from either side of the relationship.
- Both `Note` and `Feedback` attach to a `Meeting`. `Feedback` additionally stores its own direct `student` reference, so a student's feedback history can be queried without joining through `Meeting`.

Chapter 6

Modules and Features

6.1 Authentication and User Management

Every user, regardless of role, is created and authenticated through the same `User` collection and the same pair of endpoints, `POST /api/users` for registration and `POST /api/users/login` for authentication. Passwords are hashed with `bcryptjs` before being stored, so the raw password is never persisted. On successful login, the server issues a signed JWT that encodes the user's identity and role. All subsequent protected requests present this token, which is verified by the `protect` middleware and checked against permitted roles by `authorizeRoles`.

6.2 Student Module

The student dashboard is the entry point for a student's mentorship activity. From here a student can:

- Submit a mentor allocation request, optionally naming a preferred faculty member.
- View their currently assigned mentor once a request has been approved.

- Book a meeting with their mentor, specifying a date and an agenda.
- Review their complete meeting history, including past, scheduled, and cancelled meetings.
- Submit feedback, a 1 to 5 rating and an optional comment, after a completed meeting.

6.3 Faculty Module

Faculty members act as mentors and interact with the system primarily through the faculty dashboard:

- Review a list of pending mentor allocation requests addressed to them or open generally.
- Approve a request, which activates the assignment and updates the student's mentor reference.
- View a list of currently assigned mentees.
- Update their own availability so students can see when they are reachable.
- View and manage upcoming meetings, and add structured notes to a completed session.

6.4 Admin Module

The admin dashboard gives an institution-wide view of mentorship activity:

- A department-wise summary showing total students, total faculty, total and active assignments, reassigned students, and meeting counts broken down by status.
- The ability to reassign a student from one mentor to another, which is used when a faculty member becomes unavailable or a mismatch needs correcting.

6.5 Meetings, Notes, and Feedback

Meetings are the operational core of the system once a mentor is assigned:

- Meetings can be created, listed, have their notes updated, and be cancelled.
- Additional structured notes can be attached to a specific meeting through the notes module, independent of the meeting's own notes field, which allows multiple contributors to record observations over time.
- Feedback is collected once a meeting has taken place, giving the system a simple but effective quality signal per mentor and per department.

Chapter 7

Sequence Diagrams

7.1 User Login

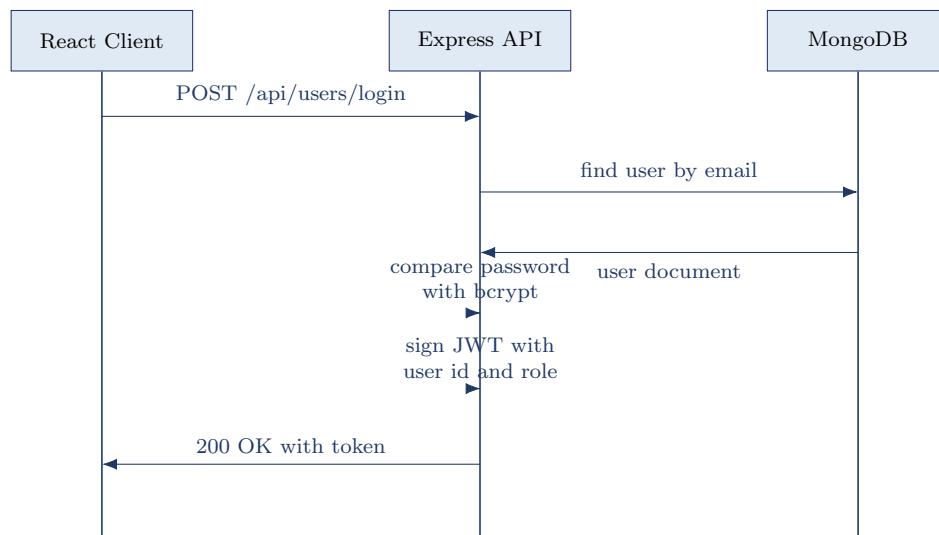


Figure 7.1: Sequence diagram for user login and JWT issuance

7.2 Mentor Request and Approval

7.3 Meeting and Feedback

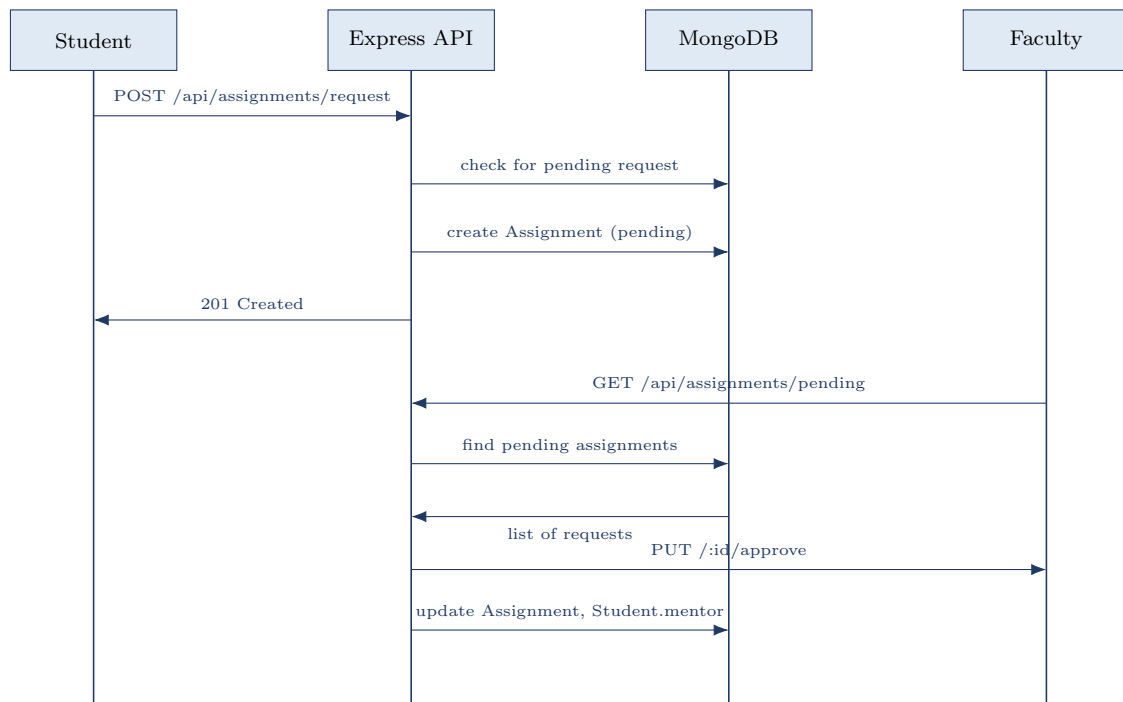


Figure 7.2: Sequence diagram for the mentor request and approval workflow

Chapter 8

API Reference

8.1 Conventions

All endpoints are prefixed with `/api`. Protected routes require an `Authorization: Bearer <token>` header carrying a valid JWT. Every response follows the shape `{ success, message, data }`, where `data` is omitted for simple confirmations.

8.2 Endpoint Summary

Method	Endpoint	Access	Description
GET	<code>/api/health</code>	Public	API health check
POST	<code>/api/users</code>	Public	Register a new user
POST	<code>/api/users/login</code>	Public	Log in, receive a JWT
GET	<code>/api/users</code>	Public	List users
POST	<code>/api/mentors</code>	Public	Create a mentor profile
GET	<code>/api/mentors</code>	Public	List mentors

Method	Endpoint	Access	Description
GET	<code>/api/mentors/:id/mentees</code>	Public	List a mentor's mentees
PUT	<code>/api/mentors/:id/availability</code>	Public	Update mentor availability
POST	<code>/api/students</code>	Public	Create a student profile
GET	<code>/api/students</code>	Public	List students
GET	<code>/api/students/:id/mentor</code>	Public	Get a student's assigned mentor
GET	<code>/api/students/:id/meetings</code>	Public	Get a student's meetings
POST	<code>/api/assignments</code>	Public	Create an assignment directly
GET	<code>/api/assignments</code>	Public	List assignments
GET	<code>/api/assignments/summary</code>	Public	Department-wise summary
POST	<code>/api/assignments/request</code>	Student	Request mentor allocation
GET	<code>/api/assignments/pending</code>	Faculty	View pending requests
PUT	<code>/api/assignments/:id/approve</code>	Faculty	Approve a mentor request
PUT	<code>/api/assignments/:id/reassign</code>	Admin	Reassign a student's mentor
POST	<code>/api/meetings</code>	Public	Schedule a meeting
GET	<code>/api/meetings</code>	Public	List meetings
PUT	<code>/api/meetings/:id/notes</code>	Public	Update meeting notes
PUT	<code>/api/meetings/:id/cancel</code>	Public	Cancel a meeting
POST	<code>/api/notes</code>	Public	Create a note
GET	<code>/api/notes</code>	Public	List notes
GET	<code>/api/notes/meeting/:meetingId</code>	Public	Notes for a specific meeting
POST	<code>/api/feedback</code>	Public	Submit feedback
GET	<code>/api/feedback</code>	Public	List feedback

Note: "Public" here means the route is not currently wrapped in the `protect` middleware in this version of the codebase. Role-gated routes, marked Student, Faculty, or Admin, are enforced with `protect` together with `authorizeRoles`.

8.3 Sample Request and Response

The listing below shows the request body and response shape for approving a mentor request, taken directly from the assignment controller.

Listing 8.1: Request

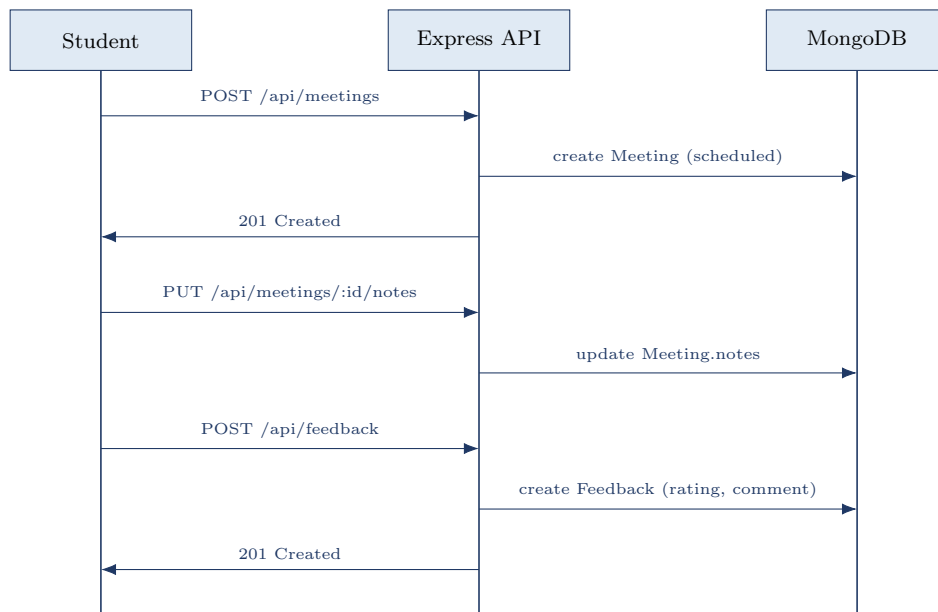


Figure 7.3: Sequence diagram for booking a meeting and submitting feedback

```

1 PUT /api/assignments/68f2c1.../approve
2 Authorization: Bearer <faculty JWT>

```

Listing 8.2: Response

```

1 {
2   "success": true,
3   "message": "Mentor request approved successfully",
4   "data": {
5     "_id": "68f2c1...",
6     "studentId": { "...": "populated student" },
7     "facultyId": { "...": "populated faculty" },
8     "status": "active",
9     "reassignedFlag": false
10  }
11 }

```

Chapter 9

Security Considerations

9.1 Implemented

- Password hashing through `bcryptjs`, so plaintext passwords are never stored in the database.
- Stateless authentication using signed JWTs, verified independently on every protected request.
- Role-based access control on the sensitive assignment operations: a student may request a mentor, only faculty may approve requests, and only an admin may reassign a student.
- CORS enabled at the API boundary for controlled cross-origin access from the frontend.

9.2 Recommended Hardening

The following items are recommended as future hardening steps before any production deployment:

- Apply the `protect` and `authorizeRoles` middleware consistently across all remaining routes; several list and create endpoints are currently open, as noted in the API reference.
- Route every controller's error handling through the centralized `errorMiddleware` for consistent, predictable error responses.
- Add request body validation, for example with `express-validator` or `zod`, on all POST and PUT endpoints.
- Remove secrets entirely from source control and rotate the JWT signing secret on a regular schedule.
- Add rate limiting on the login endpoint to reduce the risk of brute-force credential attacks.

Chapter 10

Testing

10.1 Approach

Testing during development was carried out primarily at the API level, using Postman as the client. The backend repository includes a dedicated `postman` directory containing exported resources and workspace globals, which were used to exercise every endpoint against a running instance of the API and a MongoDB Atlas development database.

10.2 What Was Tested

- Registration and login, including rejection of duplicate emails and incorrect passwords.
- Token verification, confirming that requests without a valid `Authorization` header are rejected with a 401 response.
- Role enforcement, confirming that a student token cannot call the faculty approval endpoint and that a faculty token cannot call the admin reassignment endpoint.
- The full mentor request and approval life cycle, from a student's initial request through to an active assignment and an updated `Student.mentor` reference.
- Meeting creation, note updates, and cancellation, including the effect on subsequent meeting listings.
- Feedback submission and its constraint that the rating must fall between 1 and 5.
- The department summary endpoint, cross-checked manually against known seed data to confirm the returned counts were correct.

10.3 Manual Frontend Verification

Each role's dashboard was manually walked through end to end: registering as a student, requesting a mentor, logging in as faculty to approve the request, booking and completing a meeting, submitting feedback, and finally logging in as an admin to confirm the department summary reflected the activity performed.

Chapter 11

Conclusion and Future Scope

11.1 Current Status

The project delivers a working, end-to-end mentorship platform. A student can register, request a mentor, have that request approved by faculty, book and manage meetings, and leave feedback, while an admin retains full oversight through department summaries and reassignment controls. The backend exposes a complete REST API across seven resource domains, secured with JWT authentication and role-based authorization on all sensitive operations, and the frontend provides distinct, role-appropriate dashboards for all three user types.

11.2 Key Outcomes

- A clean separation between authentication identity (**User**) and role-specific profile data (**Student**, **Mentor**), which keeps the schema easy to extend.
- A complete mentor allocation workflow, from request through approval to reassignment, modeled explicitly as an **Assignment** state machine.
- A meeting and feedback subsystem that gives the institution a lightweight but genuine quality signal on mentorship activity.
- A frontend built around role-specific dashboards rather than a single generic interface, which keeps each user's experience focused on the actions relevant to them.

11.3 Future Scope

- Email or in-app notifications for meeting scheduling and mentor request approval.
- Calendar integration for mentor availability and meeting booking.
- An analytics dashboard for admins, covering meeting frequency and feedback trends over time.
- File or document attachment support for meeting notes.
- A deployment pipeline with CI/CD and containerization to move the prototype toward production readiness.
- Completing the security hardening items listed in Chapter 9 before any real-world rollout.